

Komite - Diseno funcional de inspecciones, mantenencias y planificacion operativa

1. Objetivo

Este documento define una primera vision funcional para el modulo de inspecciones, mantenencias y planificacion operativa de Komite.

La idea nace a partir de la plantilla **Formato Programacion Mantenimientos Anuales.xlsx**, que funciona como una matriz anual de mantenencias por categoria, instalacion, tarea, periodicidad y meses programados.

El objetivo no es crear desde el primer dia un motor complejo de optimizacion, sino construir una base ordenada que permita:

- Gestionar plantillas maestras desde el backoffice de Komite.
- Permitir que cada condominio tenga su propia version de una plantilla.
- Programar mantenencias, inspecciones y tareas recurrentes.
- Asignar responsables.
- Registrar ejecucion, evidencia, atrasos y reprogramaciones.
- Preparar el terreno para una futura capa de optimizacion o replanificacion automatica.

2. Vision general

Komite deberia evolucionar hacia un sistema donde la administradora pueda transformar una planificacion teorica en una operacion diaria controlable.

Flujo conceptual:

```
Backoffice Komite
  crea plantilla base de mantenencias / inspecciones
    |
Empresa administradora / Condominio
  duplica y adapta la plantilla a su realidad
    |
Portal administrador
  programa fechas, responsables y recurrencias
    |
App mobile / Portal
  responsables ejecutan, reportan evidencia y cierran tareas
    |
Komite
  mide cumplimiento, detecta desviaciones y propone replanificacion
```

3. Alcance inicial

3.1 Incluido en una primera version funcional

- Backoffice para crear y mantener plantillas base.
- Versiones de plantilla por condominio.
- Catalogo de categorias, instalaciones/equipos y tareas.
- Periodicidad de cada tarea.
- Programacion anual o mensual.
- Generacion de eventos planificados.
- Asignacion de responsable.
- Registro de estado de ejecucion.
- Evidencias: fotos, comentarios, documentos o audio.
- Panel de seguimiento por condominio.

3.2 Fuera del alcance inicial

- Algoritmos geneticos.
- Optimizacion avanzada de rutas o tiempos.
- Prediccion automatica de atrasos.
- Reasignacion automatica sin validacion humana.
- Integraciones con calendarios externos.

Estos puntos se consideran una fase posterior, cuando el sistema ya tenga datos reales de ejecucion.

4. Lectura de la plantilla actual

La plantilla Excel analizada tiene una hoja llamada **Plan Anual**.

Estructura observada:

- Titulo: **PLAN ANUAL MANTENIMIENTO - KOMPLEMENTA**.
- Columnas base:
 - Descripcion / categoria general.
 - Instalacion o equipo.
 - Mantencion.
 - Periodicidad.
- Columnas por mes:
 - ABR, MAY, JUN, JUL, AGO, SEP, OCT, NOV, DIC, ENE, FEB, MAR.
- Las marcas **X** indican en que meses corresponde ejecutar una mantencion.

Ejemplos de categorias detectadas:

- Instalaciones electricas.
- Generador.
- Corrientes debiles.
- Agua potable y alcantarillado.
- Cierros perimetrales.
- Espacios comunes.
- Piscina.

Ejemplos de tareas:

- Revision y reapriete de automaticos y diferenciales.
- Revision y sustitucion de ampolletas o focos.
- Mantenimiento con proveedor.
- Arranque preventivo.
- Limpieza y desinfeccion de estanques.
- Revisar valvulas de corte automatico.
- Chequear funcionamiento alternado de bombas.

Esta estructura es una buena base para crear un catalogo de tareas recurrentes.

5. Conceptos funcionales principales

5.1 Plantilla base

Una plantilla base es un modelo mantenido por Komite desde el backoffice.

Debe representar buenas practicas generales, por ejemplo:

- Plan anual de mantenciones estandar.
- Checklist de inspeccion semanal.
- Checklist de sala de bombas.
- Checklist de piscina.
- Checklist de generador.
- Checklist de espacios comunes.

La plantilla base no pertenece a un condominio concreto. Es una referencia reutilizable.

5.2 Plantilla de condominio

Un condominio puede duplicar una plantilla base y adaptarla.

Ejemplos de adaptacion:

- Eliminar tareas que no aplican.
- Agregar tareas propias.
- Cambiar periodicidad.
- Cambiar meses planificados.
- Asignar responsables habituales.
- Definir proveedor externo.
- Agregar instrucciones internas.

La plantilla de condominio debe conservar referencia a la plantilla base para saber de donde viene, pero debe poder evolucionar de forma independiente.

5.3 Tarea planificada

Una tarea planificada es una ocurrencia concreta generada a partir de la plantilla.

Ejemplo:

```
Condominio: Ciudad del Encanto V
Categoria: Agua potable y alcantarillado
Instalacion: Sala de bombas
Tarea: Chequear que bombas funcionen alternadamente
Fecha programada: 2026-07-08
Responsable: Supervisor
Estado: Pendiente
```

5.4 Ejecucion

La ejecucion representa lo que realmente ocurrio.

Debe permitir registrar:

- Fecha y hora real.
- Responsable que ejecuta.
- Resultado: correcto, con observacion, no realizado, requiere accion.
- Comentarios.
- Evidencias.
- Firma o validacion si aplica.
- Nueva incidencia o ticket relacionado si se detecta un problema.

5.5 Reprogramacion

Cuando una tarea no se cumple o surge una urgencia, debe poder reprogramarse.

Motivos posibles:

- Responsable no disponible.
- Emergencia en otro condominio.
- Proveedor no asistio.
- Condiciones tecnicas impiden la ejecucion.
- Prioridad operacional.
- Tarea duplicada o innecesaria.

La reprogramacion debe quedar trazada.

6. Actores

6.1 Komite backoffice

Usuarios internos de Komite.

Responsabilidades:

- Crear plantillas base.
- Versionar plantillas.

- Desactivar plantillas antiguas.
- Definir categorias estandar.
- Mantener catalogos globales.

6.2 Empresa administradora

Usuarios del portal administrador.

Responsabilidades:

- Adoptar plantillas base.
- Adaptarlas a sus condominios.
- Programar calendario operativo.
- Asignar responsables.
- Revisar cumplimiento.

6.3 Project manager

Responsable de coordinar la operacion de varios condominios.

Responsabilidades:

- Visualizar carga de trabajo.
- Reasignar tareas.
- Resolver atrasos.
- Priorizar urgencias.
- Validar cumplimiento operativo.

6.4 Ejecutivo/a

Usuario operativo de la empresa administradora.

Responsabilidades:

- Revisar tareas asignadas.
- Registrar avance.
- Coordinar proveedores.
- Adjuntar comentarios y documentos.

6.5 Supervisor

Usuario de terreno.

Responsabilidades:

- Ejecutar inspecciones.
- Completar checklists.
- Subir fotos.
- Reportar problemas.
- Solicitar reprogramacion si corresponde.

6.6 Conserje

Usuario de apoyo operativo.

Responsabilidades:

- Registrar novedades.
 - Ejecutar tareas simples.
 - Reportar incidencias.
 - Confirmar visitas de proveedores.
-

7. Estados funcionales

7.1 Estados de una plantilla

- Borrador.
- Activa.
- Inactiva.
- Archivada.

7.2 Estados de una tarea planificada

- Pendiente.
- Programada.
- En curso.
- Realizada.
- Realizada con observacion.
- No realizada.
- Reprogramada.
- Cancelada.

7.3 Estados de validacion

- Sin validar.
 - Validada por supervisor.
 - Validada por administrador.
 - Rechazada.
 - Requiere accion.
-

8. Modelo de datos propuesto

Este modelo es orientativo. La implementacion final debe adaptarse a la arquitectura actual de Komite.

8.1 inspection_templates

Plantillas base creadas por Komite.

Campos sugeridos:

- id.
- tenant_id.

- name.
- description.
- template_type.
- version.
- status.
- source_file_name.
- created_by.
- created_at.
- updated_at.

8.2 inspection_template_sections

Agrupar tareas dentro de una plantilla.

Ejemplos: instalaciones electricas, agua potable, piscina.

Campos sugeridos:

- id.
- tenant_id.
- template_id.
- name.
- description.
- display_order.
- status.

8.3 inspection_template_items

Lineas de trabajo dentro de una plantilla.

Campos sugeridos:

- id.
- tenant_id.
- template_id.
- section_id.
- asset_name.
- task_name.
- instructions.
- periodicity.
- planned_months.
- requires_evidence.
- default_responsible_profile.
- default_duration_minutes.
- display_order.
- status.

8.4 condominium_inspection_templates

Version adaptada de una plantilla para un condominio.

Campos sugeridos:

- id.
- tenant_id.
- company_id.
- condominium_id.
- base_template_id.
- name.
- version.
- status.
- effective_from.
- effective_to.
- created_by.

8.5 condominium_inspection_items

Tareas adaptadas para un condominio.

Campos sugeridos:

- id.
- tenant_id.
- condominium_template_id.
- base_item_id.
- section_name.
- asset_name.
- task_name.
- instructions.
- periodicity.
- planned_months.
- responsible_user_id.
- responsible_profile.
- provider_id.
- estimated_duration_minutes.
- priority.
- status.

8.6 planned_operational_events

Eventos concretos generados desde una plantilla.

Campos sugeridos:

- id.
- tenant_id.
- company_id.
- condominium_id.
- condominium_template_item_id.
- title.

- description.
- planned_date.
- planned_start_time.
- planned_end_time.
- assigned_user_id.
- assigned_profile.
- priority.
- status.
- source_type.
- source_id.
- created_by.

8.7 operational_event_executions

Registro de ejecucion real.

Campos sugeridos:

- id.
- tenant_id.
- event_id.
- executed_by_user_id.
- executed_at.
- result.
- comments.
- requires_follow_up.
- related_incident_id.
- related_ticket_id.
- validation_status.
- validated_by_user_id.
- validated_at.

8.8 operational_event_evidence

Evidencias asociadas a la ejecucion.

Campos sugeridos:

- id.
- tenant_id.
- event_id.
- execution_id.
- attachment_id.
- evidence_type.
- description.
- created_by.
- created_at.

8.9 operational_reschedule_logs

Historial de reprogramaciones.

Campos sugeridos:

- id.
 - tenant_id.
 - event_id.
 - previous_date.
 - new_date.
 - previous_assigned_user_id.
 - new_assigned_user_id.
 - reason.
 - requested_by_user_id.
 - approved_by_user_id.
 - created_at.
-

9. Backoffice Komite

El backoffice debería gestionar los elementos globales que pertenecen a Komite.

Pantallas sugeridas:

9.1 Plantillas

Listado de plantillas base.

Acciones:

- Crear plantilla.
- Importar desde Excel.
- Editar plantilla.
- Duplicar plantilla.
- Versionar plantilla.
- Activar / desactivar.

9.2 Editor de plantilla

Debe permitir editar:

- Nombre.
- Descripción.
- Tipo.
- Versión.
- Categorías/secciones.
- Instalaciones/equipos.
- Tareas.
- Periodicidad.
- Meses sugeridos.
- Responsable sugerido.

- Requiere evidencia.

9.3 Importador de plantilla Excel

Debe permitir cargar una plantilla como la enviada.

Reglas iniciales:

- Cada fila con mantencion valida crea un item.
 - La columna Descripcion se interpreta como categoria.
 - La segunda columna se interpreta como instalacion/equipo.
 - La columna Mantencion se interpreta como tarea.
 - La columna Periodicidad se guarda como texto normalizado.
 - Las columnas de meses con X se guardan como meses planificados.
-

10. Portal administrador

El portal administrador deberia usar las plantillas para cada condominio.

Pantallas sugeridas:

10.1 Plantillas del condominio

Permite ver plantillas aplicadas al condominio seleccionado.

Acciones:

- Adoptar plantilla base.
- Duplicar plantilla.
- Personalizar plantilla.
- Activar plantilla.

10.2 Programacion

Vista calendario/listado de eventos.

Filtros:

- Fecha.
- Responsable.
- Estado.
- Categoria.
- Prioridad.

Acciones:

- Crear evento manual.
- Reprogramar.
- Asignar responsable.
- Cambiar prioridad.

10.3 Cumplimiento

Indicadores:

- Tareas programadas.
 - Tareas realizadas.
 - Tareas atrasadas.
 - Tareas reprogramadas.
 - Cumplimiento por responsable.
 - Cumplimiento por categoria.
-

11. App mobile

La app mobile deberia ser la herramienta principal de ejecucion en terreno.

Funciones sugeridas:

- Ver tareas asignadas del dia.
 - Ver tareas por condominio.
 - Completar checklist.
 - Adjuntar fotos.
 - Grabar audio.
 - Marcar como realizado.
 - Solicitar reprogramacion.
 - Crear incidencia desde una inspeccion.
-

12. Relacion con incidencias, tickets y comunicaciones

Este modulo no debe sustituir incidencias ni tickets.

Relacion recomendada:

- Si una inspeccion detecta un problema del condominio, puede crear una incidencia.
 - Si el problema afecta al cliente o al soporte Komite, puede crear un ticket.
 - Si la ejecucion requiere informar al comite o vecinos, puede generar una comunicacion.
 - Si una tarea se repite o no se ejecuta, puede aparecer en informes operativos.
-

13. Futura optimizacion y replanificacion

La optimizacion no deberia ser la primera fase, pero el modelo debe prepararse para ella.

Para que en el futuro tenga sentido usar algoritmos geneticos, heurísticas u optimizacion, Komite necesitara datos historicos:

- Duracion real de tareas.
- Responsables disponibles.
- Ubicacion de condominios.
- Prioridad de tareas.

- Ventanas horarias.
- Dependencias entre tareas.
- Tareas no realizadas.
- Motivos de reprogramacion.
- Carga diaria por responsable.
- Tiempo de desplazamiento.

13.1 Primera aproximacion no automatica

Antes de algoritmos complejos, se puede crear un asistente simple:

- Detectar atrasos.
- Sugerir mover tareas no urgentes.
- Alertar sobre sobrecarga de un supervisor.
- Agrupar tareas por condominio.
- Priorizar tareas criticas.

13.2 Fase de optimizacion avanzada

Cuando existan datos suficientes:

- Replanificacion diaria semiautomatica.
- Sugerencia de agenda por supervisor.
- Optimizacion por prioridad, distancia y disponibilidad.
- Simulacion de impacto ante urgencias.
- Comparacion entre plan original y plan reoptimizado.

La recomendacion es que el sistema proponga cambios, pero que un usuario humano los apruebe.

14. Fases recomendadas

Fase 1 - Base de plantillas

- Crear tablas de plantillas.
- Crear backoffice para plantillas.
- Importar Excel base.
- Permitir duplicar plantilla para condominio.

Fase 2 - Programacion por condominio

- Crear calendario/listado de eventos.
- Generar eventos desde periodicidad.
- Asignar responsables.
- Editar fechas manualmente.

Fase 3 - Ejecucion en terreno

- App mobile con tareas del dia.
- Registro de evidencias.

- Estados de ejecucion.
- Creacion de incidencias desde inspecciones.

Fase 4 - Seguimiento operativo

- Dashboard de cumplimiento.
- Alertas por atraso.
- Informes por condominio.
- Historial de reprogramaciones.

Fase 5 - Replanificacion asistida

- Sugerencias simples.
- Reglas de prioridad.
- Deteccion de sobrecarga.
- Reasignacion propuesta.

Fase 6 - Optimizacion avanzada

- Algoritmos de optimizacion.
- Simulacion diaria.
- Planificacion multi-condominio.
- Comparacion planificado vs real.

15. Riesgos y decisiones pendientes

15.1 Riesgos

- Intentar automatizar demasiado pronto.
- Crear un modelo de datos demasiado rigido.
- Confundir tareas internas con incidencias visibles para vecinos.
- Sobrecargar al supervisor con formularios largos.
- No capturar datos suficientes para futura optimizacion.

15.2 Decisiones pendientes

- Si la plantilla anual debe generar eventos mensuales, semanales o diarios automaticamente.
- Si las tareas permanentes deben aparecer todos los días o solo bajo checklist.
- Si un responsable puede ser usuario concreto o perfil.
- Si los proveedores externos deben gestionarse como entidad propia.
- Si cada ejecucion requiere validacion del administrador.
- Si las mantenciones deben ser visibles para comite/vecinos o solo para la empresa.

16. Recomendacion funcional

La recomendacion es comenzar con una version sencilla y robusta:

1. Backoffice crea plantillas base.

2. Condominio adopta y adapta una plantilla.
3. El sistema genera eventos planificados.
4. Responsables ejecutan y suben evidencia.
5. El portal muestra cumplimiento y atrasos.

Solo cuando existan datos reales de varios meses, tiene sentido avanzar hacia optimizacion automatica.

Esta estrategia evita construir un motor complejo antes de conocer como trabajan realmente los supervisores, administradores y conserjes en el dia a dia.